

3 Solutions logicielles au problème d'exclusion mutuelle

Dans le cadre de ce cours, nous nous limitons aux algorithmes qui synchronisent seulement deux processus.

Solution par attente active

Tentative 1 :

Le principe de cette solution est basé sur l'utilisation d'une variable booléenne appelée **occupe** qui indique si l'un des processus s'exécute en section critique.

Initialement, *occupe* est positionné à faux.

bool occupe = false ;	
void *P0(void *arg) { while (true) { while (occupe) ; occupe = true ; /* Section Critique */ occupe = false ; /* section non-critique */ } }	void *P1(void *arg) { while (true) { while (occupe) ; occupe = true ; /* Section Critique */ occupe = false ; /* section non-critique */ } }

Mais, cette solution est incorrecte comme le montre le scénario suivant.

P_0 : lire *occupe* à faux

P_1 : lire aussi *occupe* à faux

P_1 : mettre *occupe* à vrai

P_1 : entre en section critique

P_0 : mettre aussi *occupe* à vrai

P_0 : entre aussi en section critique.

Les deux processus P0 et P1 entre en même temps en section critique.

Tentative 2 :

Cette solution utilise une variable entière **tour** qui indique l'identité du processus qui a l'autorisation d'accéder à la section critique. Ainsi P_i peut accéder à la section critique seulement si $tour = i$.

Notre solution garantit qu'une seule tâche peut se trouver en section critique. Cependant elle ne satisfait pas la contrainte de **progression**. Les processus doivent exécuter leur section critique en alternance.

int tour = 0; // ou 1	
void *P0(void *arg) { while (true) { while (tour != 0) ; /* Section critique */ tour = 1; /* section non-critique */ } }	void *P1(void *arg) { while (true) { while (tour != 1) ; /* Section Critique */ tour = 0; /* section non-critique */ } }

Un processus peut donc attendre pour entrer en section critique alors que celui auquel le tour lui revient ne l'utilise pas et n'a peut-être plus l'intention de l'utiliser. Le plus lent des deux processus impose son rythme à l'autre. De plus, si un des deux processus disparaît, l'autre attendra indéfiniment.

Tentative 3 :

Dans cette tentative, la variable **tour** de la tentative précédente est remplacée par un vecteur de booléen **etat** initialisé à false. Cette tentative se base sur l'état du processus plus tôt que son identité. C'est-à-dire que **etat[i]** est à **true** si **P_i** est en section critique.

bool etat[2] = {false,false};	
void *P0(void *arg) { while (true) { while (etat[1]) ; etat[0] = true ; /* Section Critique */ etat[0] = false ; /* section non-critique */ } }	void *P1(void *arg) { while (true) { while (etat[0]) ; etat[1] = true ; /* Section Critique */ etat[1] = false ; /* section non-critique */ } }

Le scenario suivant montre que cette tentative ne respecte pas l'exclusion mutuelle puisque les deux processus entre en SC en même temps.

P_0 : exécuter la boucle et trouver *etat*[1] à false
 P_1 : exécuter la boucle et trouver aussi *etat*[0] à false
 P_0 : mettre *etat*[0] à true et entrer en section critique
 P_1 : mettre *etat*[1] à true et entrer aussi en section critique.

Tentative 4 :

Dans la tentative précédente, l'un des processus peut vérifier l'état de l'autre avant que ce dernier ait l'occasion de modifier son état. Cette tentative propose de modifier l'état de processus avant de faire la vérification avec la boucle. C'est-à-dire que *etat*[*i*] est à *true* si P_i veut accéder à la section critique.

bool <i>etat</i> [2] = { false , false };	
void *P0(void *arg) { while (true) { <i>etat</i> [0] = true ; while (<i>etat</i> [1]) ; /* Section Critique */ <i>etat</i> [0] = false ; /* section non-critique */ } }	void *P1(void *arg) { while (true) { <i>etat</i> [1] = true ; while (<i>etat</i> [0]) ; /* Section Critique */ <i>etat</i> [1] = false ; /* section non-critique */ } }

Cette modification garantit l'exclusion mutuelle entre les deux processus, mais elle engendre un autre problème comme le montre le scenario suivant :

P_0 : mettre *etat*[0] à true
 P_1 : mettre *etat*[1] à true
 P_0 : exécuter la boucle et se bloque
 P_1 : exécuter la boucle et se bloque aussi

Si les deux processus progressent simultanément dans leur protocole d'entrée, ils se bloquent mutuellement. Ceci est provoqué du fait que P_i indique son intention d'entrer en section critique sans savoir l'intention de l'autre.

Tentative 5 :

Pour résoudre le problème de la tentative précédente, cette tentative oblige un processus à annuler temporairement son intention d'entrer dans sa section critique si l'autre processus veut y entrer également.

bool <i>etat</i> [2] = { false , false };	
void *P0(void *arg) { while (true) { <i>etat</i> [0] = true ; while (<i>etat</i> [1]) { <i>etat</i> [0] = false ; while (<i>etat</i> [1]) ; <i>etat</i> [0] = true ; } /* Section Critique */ <i>etat</i> [0] = false ; /* section non-critique */ } } }	void *P1(void *arg) { while (true) { <i>etat</i> [1] = true ; while (<i>etat</i> [0]) { <i>etat</i> [1] = false ; while (<i>etat</i> [0]) ; <i>etat</i> [1] = true ; } /* Section Critique */ <i>etat</i> [1] = false ; /* section non-critique */ } } }

Cette tentative assure l'exclusion mutuelle entre les processus, mais un interblocage est possible si les deux processus exécutent leurs protocoles d'entrée en alternance comme le montre le scénario suivant :

*P*₀ : mettre *etat*[0] à true
*P*₁ : mettre *etat*[1] à true
*P*₀ : exécuter la boucle externe et trouvee *etat*[1] à true
*P*₁ : exécuter la boucle externe et trouve *etat*[0] à true
*P*₀ : mettre *etat*[0] à false
*P*₁ : mettre *etat*[1] à false
*P*₀ : exécuter la boucle interne et trouver *etat*[1] à false
*P*₁ : exécuter la boucle interne et trouver *etat*[0] à false
*P*₀ : mettre *etat*[0] à true
*P*₁ : met *etat*[1] à true

Les deux processus continueront de faire ce traitement indéfiniment.

Algorithme de Dekker

C'est la première solution correcte du problème d'exclusion mutuelle. Elle est basée sur les propositions faite dans la deuxième et la cinquième tentative respectivement.

bool etat[2] = { false , false }; int tour = 0; // ou 1	
void *P0(void *arg) { while (true) { etat[0] = true ; while (etat[1]) if (tour == 1) { etat[0] = false ; while (tour == 1) ; etat[0] = true ; } /* Section Critique */ tour = 1; etat[0] = false ; /* section non-critique */ } }	void *P1(void *arg) { while (true) { etat[1] = true ; while (etat[0]) if (tour == 0) { etat[1] = false ; while (tour == 0) ; etat[1] = true ; } /* Section Critique */ tour = 0; etat[1] = false ; /* section non-critique */ } }

Cette solution utilise la cinquième tentative et règle le problème d'interblocage par l'utilisation de la variable *tour* de la deuxième tentative qui impose un ordre d'accès à la section critique sur les deux processus. En cas de conflit, l'un des processus se renonce temporairement pour donner l'accès à l'autre processus.

Démonstration

Montrons que l'algorithme de Dekker assure les conditions de l'exclusion mutuelle.

1-Exclusion Mutuelle :

Supposons que P_0 et P_1 sont tous les deux en SC. Dans ce cas, $etat[i] = \text{true}$ pour $i = 0$ et 1 . Donc, les deux processus n'ont pas pu entrer en même temps car aucun n'aurait pu sortir de leur boucle externe $\Rightarrow$ l'un des processus est entrée avant l'autre.

Soit P_0 ce processus. Alors P_0 a trouvé $etat[1]$ à *false*. Puisque seule P_1 peut modifier $etat[1]$ et comme P_1 ne vérifie $etat[0]$ que si $etat[1]$ est à *true*, alors deux cas sont possible :

- 1- Si P_1 se trouve dans sa boucle interne, il n'en sortira que lorsque P_0 exécute son protocole de sortie.
- 2- Si P_1 se trouve en section non-critique, il bouclera dans son protocole d'entrée s'il veut accéder à la section critique.

Un raisonnement pareil peut être fait dans le cas où P_1 entre en SC avant P_0 .

La propriété d'exclusion mutuelle est donc montrée.

2-Progression :

Supposons qu'un seul processus veut entrer en section critique. Alors, il passe directement en SC quel que soit la valeur de *tour*.

Supposons maintenant que les deux processus veulent entrer en SC et que $tour=0 \Rightarrow$ Deux cas sont possibles.

- 1- Si P_0 trouve $etat[1] = false$, alors il entre directement en SC.
- 2- Si P_0 trouve $etat[1] = true$, alors P_0 attend dans sa boucle externe jusqu'à ce que $etat[1] = false$. Pendant ce temps, P_1 attend dans sa boucle interne jusqu'à ce que $tour= 1$. Mais auparavant, P_1 met $etat[1]= false$, ce qui permet à P_0 de sortir de sa boucle et d'entrée en SC.

Alors P_0 entre en SC avant P_1 car la valeur de *tour* lui donne avantage. Si $tour= 1$, l'inverse se produit et P_1 entre avant T_0 .

Donc pas d'interblocage.

3-L'équité :

Montrons que le risque de famine repose uniquement sur celui du matériel.

Une famine peut se produire lorsque P_i trouve sans cesse $etat[j]$ à faux et entre en SC alors que P_j est incapable de mettre $etat[j]= true$ empêché par les lectures répétitives de cette variable par P_i .

L'algorithme se base sur le matériel pour l'équité des requêtes mémoires. L'algorithme résout donc les conflits en un temps fini.

Algorithme de Peterson

Une dernière solution au problème d'exclusion mutuelle par variables partagées est proposée par Peterson (1981). Elle est très simple et vérifie les propriétés attendues d'une solution d'exclusion mutuelle.

L'algorithme utilise une variable *tour* et un tableau de booléen *intention[2]* comme variable partagées.

bool intention[2] = { false , false }; int tour = 0; // ou 1	
void *P0(void *arg) { while (true) { intention[0] = true ; tour = 1; while (intention[1] && tour == 1) ; /* Section Critique */ intention[0] = false ; /* section non-critique */ } }	void *P1(void *arg) { while (true) { intention[1] = true ; tour = 0; while (intention[0] && tour == 0) ; /* Section Critique */ intention[1] = false ; /* section non-critique */ } }

Démonstration

Montrons que l'algorithme de Peterson assure les conditions de l'exclusion mutuelle.

1-Exclusion Mutuelle

Lorsque le processus P_i veut accéder à la SC, il met $intention[i] = \text{true}$. La variable *tour* départage les deux processus si ceux-ci veulent accéder en même temps à la SC.

Pour vérifier que l'exclusion mutuelle est garantie, supposons que P_0 et P_1 sont tous les deux dans leurs SCs. Dans ce cas, $intention[i] = \text{true}$ pour $i = 0$ et 1 . Mais ces processus n'ont pas pu sortir de leurs boucles en même temps car *tour* ne peut prendre simultanément la valeur 0 et 1. L'un des processus est donc entré en SC avant l'autre.

Supposons que c'est P_i . Alors P_i a évalué la condition de sa boucle pendant que P_j exécutait $tour = i$. Mais à ce moment, P_j trouve $intention[i] = \text{true}$ et $tour = i$ lorsqu'il évalue la condition de sa boucle. Cette condition sera évaluée à *true* car ces variables gardent leurs valeurs tant que P_i se trouve en SC, ce qui préserve l'exclusion mutuelle.

2-Progression

Un processus P_i ne peut être bloqué dans son protocole d'entrée que s'il trouve sans cesse $intention[j] = \text{true}$ et $tour = j$.

- Si P_j ne veut pas entrer à la SC, alors $intention[j] = \text{false}$ et P_i ne peut pas être bloqué.
- Par contre si $intention[j] = \text{true}$, P_j doit aussi être dans sa boucle pour qu'il ait interblocage. Mais à ce moment, la variable $tour = 0$ ou 1 , favorisera l'un des deux processus.

Donc pas d'interblocage.

3-L'équité

Pour montrer que l'algorithme est équitable, il suffit de montrer que si P_i est dans sa boucle d'attente et que P_j est dans sa SC, P_i entrera en SC avant P_j si celui-ci veut à nouveau entrer en SC. Lorsque P_j quitte son protocole de sortie, il met $intention[j] = false$ et P_i peut accéder à son tour à la SC. Si P_j veut à nouveau entrer en SC, il met $intention[j] = true$ et $tour = i$. Ainsi P_j est bloqué dans sa boucle d'attente tant que P_i n'accède pas à la SC.

4 Solutions matérielles au problème d'exclusion mutuelle

4.1 Masquage des interruptions

Pour assurer l'exclusion mutuelle au niveau de l'exécution d'une section critique, un processus peut :

- Masquer les interruptions au niveau de protocole d'entrée en SC
- Libérer (démasquer) les interruptions au niveau de protocole de sortie de SC

P_i :

```
while(1) {  
    .....  
    Masquer les interruptions  
    SC  
    Démasquer les interruptions  
    .....  
}
```

Dans ce cas, l'exécution de la SC ne peut pas être interrompue ce qui garantit l'exclusion mutuelle.

Cette solution est facile à généraliser mais présente les inconvénients suivants :

- Elle influence la politique d'allocation du processeur.
- En cas de problème dans l'exécution de la SC, le système risque de se bloquer indéfiniment.
- Dans le cas d'un système multiprocesseur, le masquage des interruptions concerne généralement un seul processeur, ce qui n'empêche pas un second processus qui s'exécute sur un autre processeur d'entrer en SC.

4.2 Utilisation d'instructions machine spéciales

Plusieurs instructions spéciales ont été proposées pour régler le problème d'exclusion mutuelle. Il s'agit d'instructions machines qui accomplissent deux actions de façon atomique (sans interruption).

a) L'instruction **Test and Set**

Cette instruction peut être définie comme suit :

```
int TestSet(int *i) {
    if (*i == 0) {
        *i = 1;
        return 1;
    }
    else
        return 0;
}
```

La fonction **TestSet** correspond à une instruction machine et elle est exécutée de façon atomique, de ce fait elle n'est pas sujet aux interruptions.

L'utilisation de cette instruction pour réaliser une exclusion mutuelle entre N processus peut être illustrée comme suit :

<pre>int N = ... int verrou ;</pre>	
<pre>void P(int i) { // i représente le n° de processus while (true) { while (!TestSet(&verrou)); /* Section Critique */ verrou = 0 ; } }</pre>	<pre>int main() { verrou =0; parbegin(P(0), P(1), ..., P(N)) ; return 0 ; }</pre>

Seul le processus qui trouve (verrou = 0) au moment de l'exécution de TestSet entre en SC, tous les autres processus vont attendre (attente active).

*b) L'instruction **Exchange***

Elle peut être définie comme suit :

```
void Exchange(int *a, int *b){  
    int tmp ;  
    tmp = *a ;  
    *a = *b ;  
    *b = tmp ;  
}
```

Avantage :

- Applicable à n'importe quel nombre de processus sur un système monoprocesseur ou multiprocesseur (partageant une mémoire commune).
- Solution simple à réaliser et à vérifier
- Solution utilisable dans le cas de plusieurs ensembles de sections critiques, chaque ensemble de SC aura sa propre variable.

Conclusion

L'utilisation de l'attente active réduit le rendement du processeur ; pendant qu'un processus attend l'accès à sa section critique il continue à consommer du temps processeur.